

LECTURE 01

Git, *explained* (properly).

A version-control system, a collaboration platform, and the muscle memory that gets you hired.

`git` + `github` ← *two different things!*

THE PROBLEM

Two things break every team without version control.

01

Conflicts.

You edit `App.js`. Your teammate edits `App.js`. Whose version wins?
final_v3_FINAL.js is not a strategy.

02

Lost history.

You ship a bug Friday. The fix is hiding inside last Tuesday's commit.
You need a time machine, not a backup.

→ *git solves both, on one cheap data structure.*

COMMON CONFUSION

Git is not GitHub.

GIT

A command-line **tool** on your laptop.

Tracks file changes. Works offline. Free, open-source. Created by Linus Torvalds in 2005.

GITHUB (ALSO GITLAB, BITBUCKET...)

A **website** that hosts git repos and adds collaboration on top.

Pull requests, issues, code review, CI, the social layer.

you can use git without github. you cannot use github without git.

WHAT IT REALLY IS

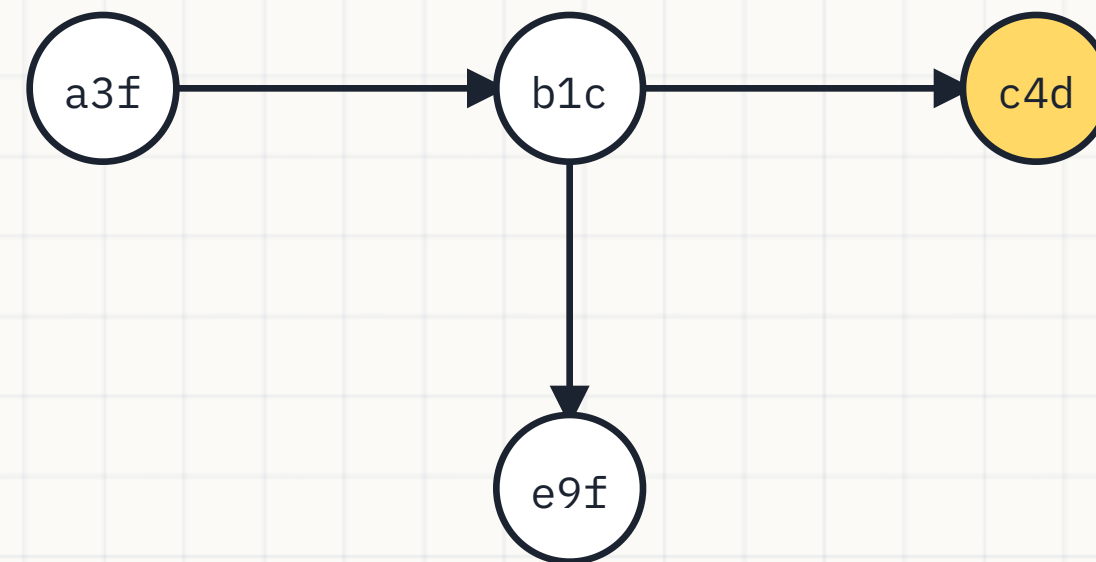
A database of *snapshots*.

Forget "version control" for a sec.

Git is a content-addressed database. You hand it files; it gives you back a hash. Same files = same hash, forever.

A **commit** bundles a tree of files with a parent pointer. Strings of commits make a graph. The graph IS your history.

that's the whole thing. everything else is UI.



each circle = a snapshot

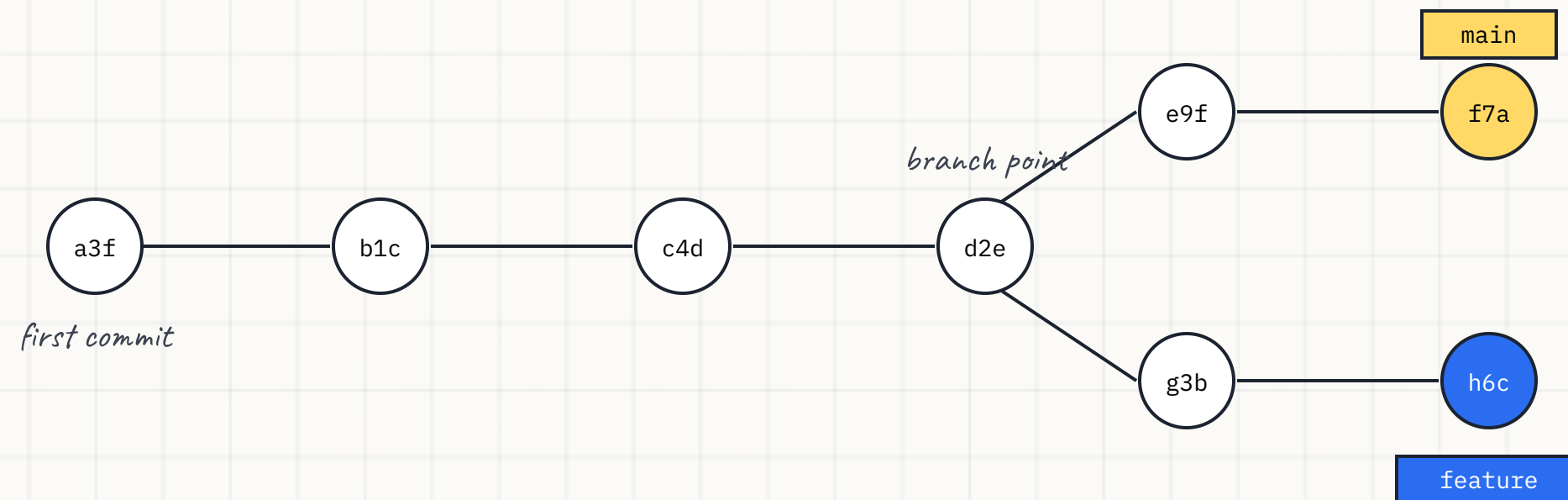
SECTION 02

The mental model.

Internalize this. Every command becomes obvious.

THE GRAPH

Commits form a graph.



arrows point to PARENTS. branches are just labels on the latest commit.

MENTAL MODEL

What's in one commit?

- ✓ **A snapshot** of every tracked file — not a diff!
- ✓ **A parent pointer** — the commit before this one
- ✓ **Author + timestamp**
- ✓ **A message** — what & why
- ✓ **A SHA hash** — content-addressed identity

change anything → new hash. that's how integrity works.

```
$ git cat-file -p HEAD
tree    8b1d4e... # files snapshot
parent  c4d3f2... # prev commit
author  Alex <a@x.io> 1714780800
committer Alex <a@x.io> 1714780800

fix: redirect to /login when
    token expired
```

MENTAL MODEL

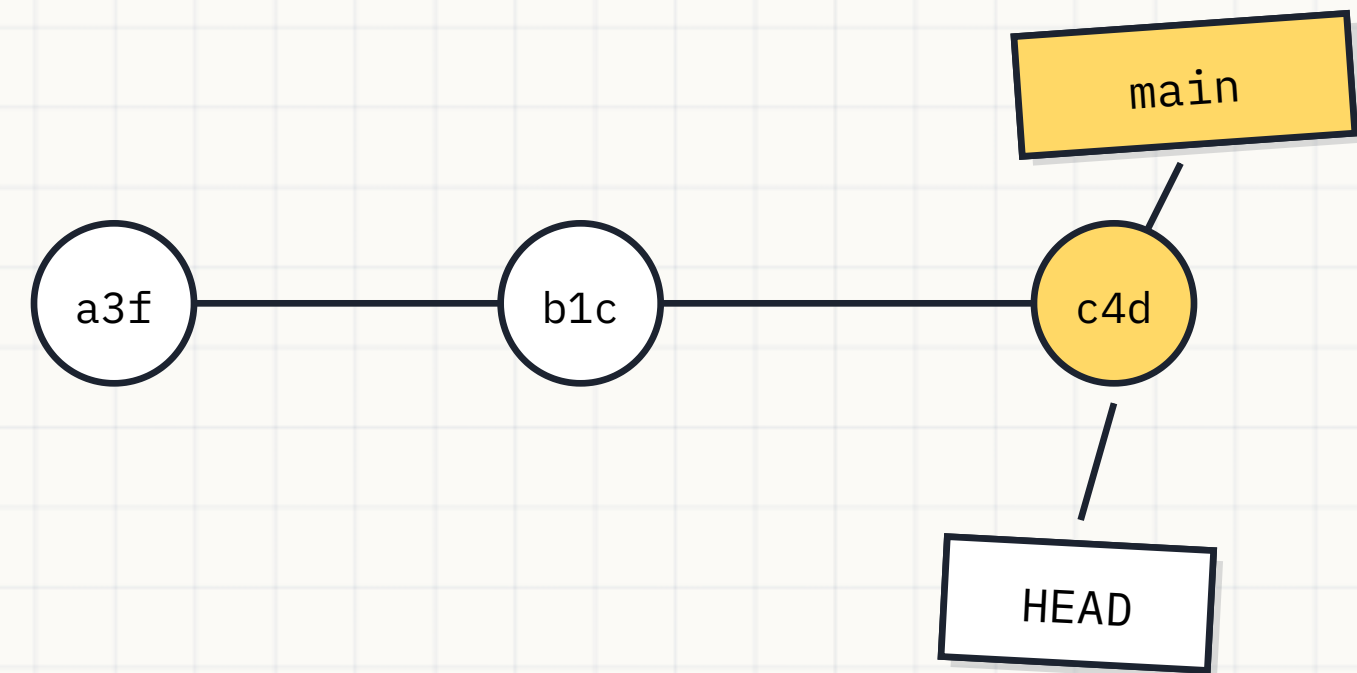
A branch is a *sticky note*.

Literally: a 40-character hash, in a text file at `.git/refs/heads/<name>`.

When you commit on a branch, the sticky moves to the new commit. **That's it.**

HEAD is the sticky that says "you are here."

cheap to make. cheap to throw away.



THE THREE TREES

Files live in three places.

01

Working dir

What you see in your editor. Files on disk. Real, mutable, messy.

"the kitchen"

02

Staging area

What you've marked for the NEXT commit. A queue of changes.

"the plate"

03

Repository

Committed history. Permanent. The graph of snapshots.

"the cookbook"

90% of beginner confusion: not knowing which tree you're in. → `git status` tells you.

DISTRIBUTED

Every laptop has the full repo.



no master copy. push & pull just sync graphs.

SECTION 03

Setup, once.

Five minutes that pay back for years.

FIRST-TIME CONFIG

Tell git who you are.

```
• • •  
# identify yourself – goes in every commit  
$ git config --global user.name "Alex Chen"  
$ git config --global user.email "a@x.io"  
  
# default branch name on new repos  
$ git config --global init.defaultBranch main  
  
# your favorite editor for commit messages  
$ git config --global core.editor "code --wait"  
  
# nicer diff coloring  
$ git config --global color.ui auto  
  
# check it  
$ git config --list
```

Once per machine. --global means: for all repos.

Use the same email as your GitHub account so commits are attributed to you on the website.

AUTH TO GITHUB

SSH keys, not passwords.

```
• • •
# 1. generate a key (press enter through prompts)
$ ssh-keygen -t ed25519 -C "a@x.io"

# 2. copy the PUBLIC key (.pub!) to clipboard
$ cat ~/.ssh/id_ed25519.pub | pbcopy

# 3. paste into github.com/settings/keys
# 4. test
$ ssh -T git@github.com
Hi alexc! You've successfully authenticated...

# now clone over SSH (note the 'git@' prefix)
$ git clone git@github.com:org/repo.git
```

HTTPS works but you'll fight token prompts forever. Set this up day one.

Public key — share this. Goes on GitHub.

Private key — never share. Stays on your laptop.

.pub = public. without .pub = private.

SECTION 04

The daily loop.

90% of git, every day, forever.

THE LOOP

Edit • Stage • Commit • Push.



internalize this rhythm. it's the heartbeat of every job.

GIT STATUS

The most-typed command.

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  modified:   src/auth.ts

Changes not staged for commit:
  modified:   README.md

Untracked files:
  notes.txt
```

Reads like a sentence. Tells you:

- ✓ Which branch
- ✓ What's staged (green)
- ✓ What's unstaged (red)
- ✓ What's untracked
- ✓ What to do next

stuck? type git status. still stuck? type it again.

GIT DIFF

See what changed.

THREE FLAVORS

```
git diff
working dir vs staging

git diff --staged
staging vs last commit

git diff main
this branch vs main
```

```
$ git diff
diff --git a/auth.ts b/auth.ts
@@ -12,6 +12,9 @@
     function login(user) {
-       return token(user)
+       if (!user) return null
+       const t = token(user)
+       return t
     }
 }
```

read the diff before every commit. catches embarrassing typos.

GIT ADD

Stage what goes in the next commit.

```
# single file
$ git add src/auth.ts

# everything in current dir & below
$ git add .

# by pattern
$ git add "*.ts"

# interactive — pick chunks within a file ✂
$ git add -p
Stage this hunk [y,n,q,a,d,s,e,]?

# oops, unstage
$ git restore --staged src/auth.ts
```

Pick which changes go into the next commit.

You can stage some changes and leave others — even pieces of one file.

add -p is how pros make atomic commits.

GIT COMMIT

Snapshot the staged stuff.

```
# quick – message inline
$ git commit -m "fix: redirect on expired token"

# skip 'add' for already-tracked files
$ git commit -am "fix: typo"

# open editor for a longer message
$ git commit

# amend – edit the LAST commit
$ git commit --amend
# forgot to add a file?
$ git add forgotten.ts
$ git commit --amend --no-edit
```

Each commit is a checkpoint you can come back to.

Commit small, commit often. Easier to review, easier to revert.

--amend rewrites history. only on commits you haven't pushed!

COMMIT MESSAGES

Write to your future self.

BAD

```
stuff  
fix  
wip wip wip  
asdf  
final fix v3
```

GOOD

```
fix: redirect to /login on expired token  
feat(auth): add Google OAuth flow  
refactor: extract validation helpers  
docs: clarify env-var setup
```

ANATOMY

type: short summary in imperative mood, ≤ 50 chars

Body explaining **WHY** not what (the diff shows what).
Wrap at 72 chars. Reference issues with [#42](#).

PUSH • PULL • FETCH

Sync with the remote.

```
• • •  
# upload your commits to origin  
$ git push  
  
# first push of a new branch — set upstream  
$ git push -u origin feature/login  
  
# download & merge in one step  
$ git pull  
  
# download WITHOUT merging — inspect first  
$ git fetch  
$ git log origin/main      # peek  
$ git merge origin/main    # now merge  
  
# pull = fetch + merge
```

pull before you start work.

push when you're done.

Pull first prevents the most common conflict scenario: you and a teammate diverged from the same starting point.

SECTION 05

Branching.

Where git earns its keep.

BRANCHES

Create. Switch. Throw away.

```
• • •  
# list branches (* = current)  
$ git branch  
* main  
  feature/login  
  
# create + switch — the modern way  
$ git switch -c feature/signup  
  
# just switch  
$ git switch main  
  
# quick toggle to previous branch  
$ git switch -  
  
# delete (after merging)  
$ git branch -d feature/login  
  
# delete remote branch  
$ git push origin --delete feature/login
```

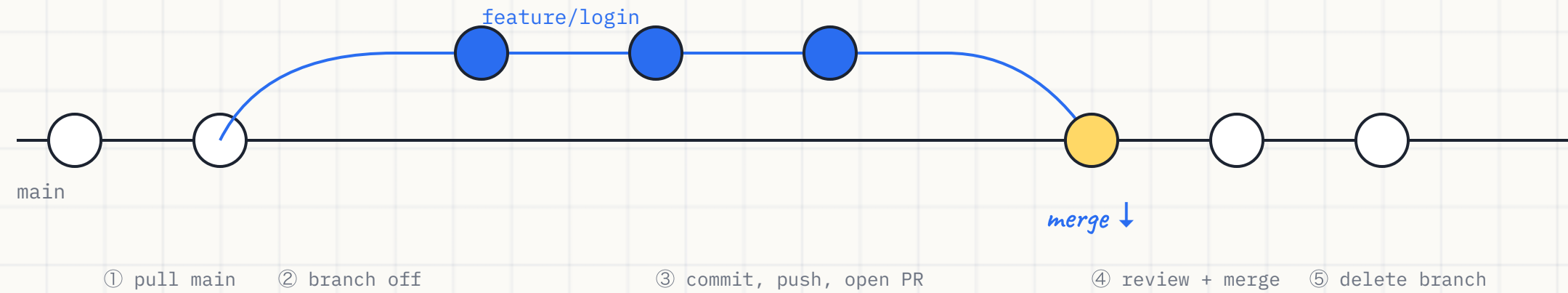
switch is the modern command.

checkout is the old one — does too many things; the muscle memory still exists in the wild.

naming: feature/x, fix/y, chore/z team-readable.

THE TEAM RHYTHM

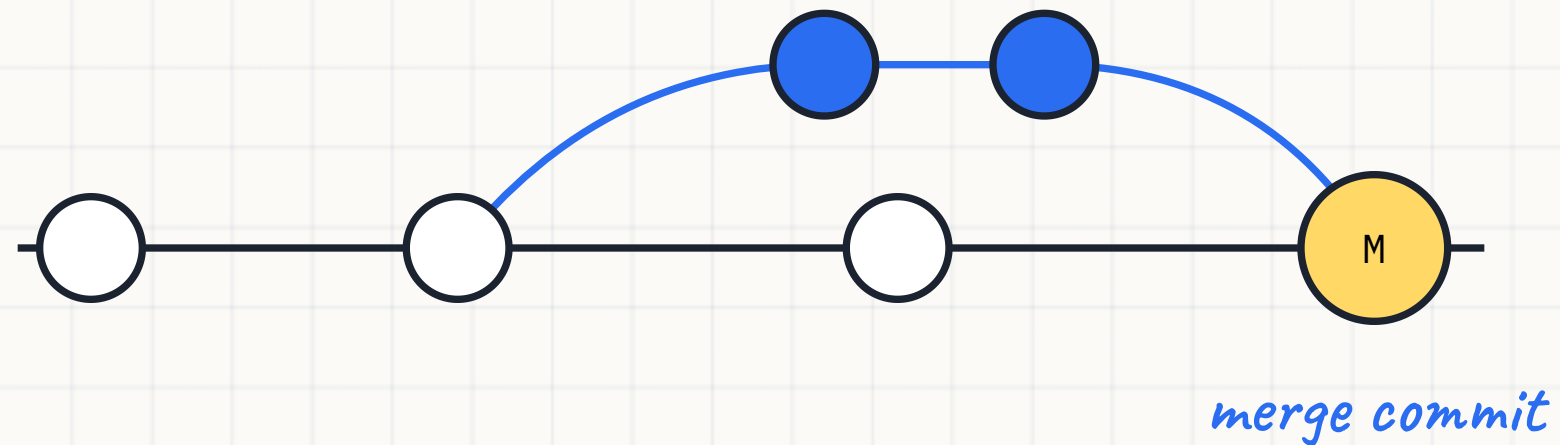
The feature-branch workflow.



main stays green. all work happens on branches. every modern team works this way.

GIT MERGE

Merge: combine, preserve history.



```
# on main, pull in feature
$ git switch main
$ git pull
$ git merge feature/login
Merge made by 'ort' strategy.
```

If main hasn't moved, it's a **fast-forward** — main just slides up. If it has, you get a **merge commit** with two parents.

GIT REBASE

Rebase: replay, keep linear.



linear — no merge commit

```
# on feature, replay onto main
$ git switch feature/login
$ git rebase main

# interactive — squash, reorder, edit
$ git rebase -i HEAD~3

# golden rule
⚠ never rebase commits you've shared.
```

merge = preserves history exactly. rebase = rewrites for cleanliness. both produce the same code.

CONFLICTS

When git can't auto-merge.

• • •

```
$ git merge feature/login
CONFLICT (content): Merge conflict in auth.ts
Automatic merge failed; fix conflicts, then commit.
```

```
# open the file – you'll see this:
```

```
function login(user) {
```

```
<<<<<<< HEAD
```

```
  return token(user)
```

```
=====
```

```
  return jwt.sign(user)
```

```
>>>>>>> feature/login
```

```
}
```

```
# 1. pick the right code, delete the markers
```

```
# 2. then:
```

```
$ git add auth.ts
```

```
$ git commit
```

```
# or, if rebasing: git rebase --continue
```

Same lines changed in both branches → git stops and asks you to decide.

HEAD = your current branch.

Below === = the branch you're merging in.

it's not scary once you've done it twice.

Bail out: `git merge --abort`

SECTION 06

Undo & history.

Almost everything is recoverable.
Git's actual superpower.

UNDO LEVELS

Different tool per layer.

WORKING DIR

Discard local edits

`git restore file.ts`

Throws away changes since last commit. Permanent — no undo.

STAGING

Unstage

`git restore --staged file.ts`

Moves the file from staged → unstaged. Edits stay intact.

LAST COMMIT

Edit message / add forgot

`git commit --amend`

Replaces the last commit. Local only — never amend after pushing.

PUSHED COMMIT

Revert (don't reset)

`git revert <sha>`

Adds a new commit that undoes the bad one. Safe for shared branches.

RESET VS REVERT

Two ways to undo a commit.

GIT RESET — REWRITES HISTORY

Moves the branch label backward. The commit "never happened."

```
git reset --soft HEAD~1
```

undo commit, keep changes staged

```
git reset --hard HEAD~1
```

⚠ undo commit AND lose changes

local only.

GIT REVERT — PRESERVES HISTORY

Adds a NEW commit that undoes the bad one. History stays honest.

```
git revert <sha>
```

creates an inverse commit

safe on shared branches. always.

GIT STASH

Park changes for later.

```
# mid-task — boss says "hotfix this other branch"
$ git stash
Saved working directory and index state WIP...

$ git switch hotfix/critical
# ...do the hotfix...
$ git switch -

# bring your work back
$ git stash pop

# inspect what's stashed
$ git stash list
stash@{0}: WIP on feature/login: a3f...

# with a name (highly recommended)
$ git stash push -m "login form half done"
```

A clipboard for uncommitted changes.

Switch branches without committing half-finished work.

use sparingly. unnamed stashes get forgotten.

GIT LOG

Browse history.

```
# default - verbose
$ git log

# compact - one line per commit ✂
$ git log --oneline
a3f9c2 fix: redirect on expired token
b1c7d4 feat: add login form
c4d2e1 chore: bump deps

# with the graph
$ git log --oneline --graph --all
* a3f9c2 (HEAD -> main) fix: ...
| * 8e2d31 (feature/login) wip
|/
* b1c7d4 feat: add login form

# filters
$ git log --author="Alex"
$ git log --since="2 weeks ago"
$ git log -- src/auth.ts      # history of one file
```

The graph IS your project history.

Alias these. You'll use them daily:

```
alias gl="git log --oneline --graph --all"
```


< THE SAFETY NET

reflog — git remembers.

...

```
# every move HEAD made — last 90 days
$ git reflog
a3f9c2 HEAD@{0}: reset: moving to HEAD~3
8e2d31 HEAD@{1}: commit: feature work
b1c7d4 HEAD@{2}: checkout: from main
c4d2e1 HEAD@{3}: pull: Fast-forward

# oh no — I reset and lost my commits
$ git reset --hard 8e2d31
HEAD is now at 8e2d31 feature work

# crisis averted ✓
```

Even commits with no branch pointing at them stick around for ~90 days.

Lost something? It's in reflog.

this command has saved more careers than any other.

SECTION 07

GitHub.

The collaboration layer on top.

WHAT GITHUB ADDS

A web UI plus collaboration tools.

01
Browse code

Read files, follow refs, search across repos. Better than your editor for unfamiliar code.

02
Pull requests

Code review UI. Inline comments, approvals, merge button.

03
Issues

The team’s todo list. Bugs, features, tasks. Linkable, assignable.

04
Actions (CI)

Run tests, lint, build, deploy on every push. Green check before merge.

05
Releases

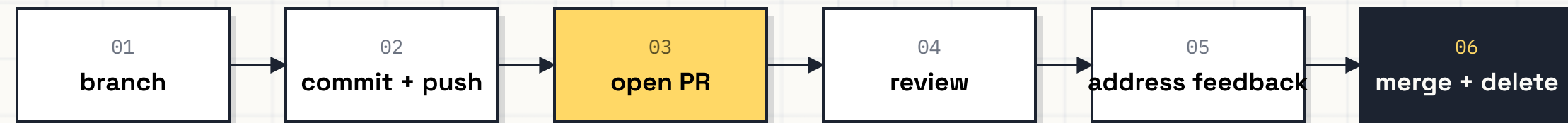
Tag versions. Attach binaries. Generate changelogs.

06
Source of truth

The shared remote. Where main lives. The team’s reality.

PULL REQUEST LIFECYCLE

The unit of collaboration.



this is the rhythm at every modern company. learn it.

ANATOMY OF A PR

What it looks like.

OPEN

feat(auth): redirect to /login on expired token

#247

AC

alex wants to merge 3 commits into main from feature/expired-token

✓ ci/tests ✓ ci/lint ✓ ci/typecheck

3 checks passed

JS

jsmith approved these changes

2 hr ago

+ 47 additions, - 12 deletions across 4 files

▸ Merge pull request

title, description, diff, checks, reviewers, comments – all in one place.

GREAT PR DESCRIPTIONS

Save the reviewer ten minutes.

What

Redirects users to /login when their JWT expires mid-session, instead of showing a 500.

Why

Closes #142. Users were getting a confusing error state and reporting it as a "the app is broken" bug.

How to test

1. Log in
2. In devtools, set token expiry to past date
3. Refresh – should redirect to /login

Screenshots

[before.png] [after.png]

- ✓ **What** changed
- ✓ **Why** — link the issue
- ✓ **How to test** — exact steps
- ✓ **Screenshots** if UI
- ✓ **Risks / rollout notes** if any

"closes #142" auto-closes the issue when merged.

CODE REVIEW

Review the change, not the person.

AS REVIEWER

- ✓ Be specific. Link to lines.
- ✓ Ask, don't demand. "What if X were null here?"
- ✓ Praise the good parts. Yes, really.
- ✓ Distinguish blockers from nits. `nit:` prefix is your friend.
- ✓ Approve when ready. Don't sit on it.

code review is the conversation. the code is just the topic.

AS AUTHOR

- ✓ Respond to every comment.
- ✓ "Done" or "won't fix because..." — both fine.
- ✓ Push fixes as new commits during review.
- ✓ Re-request review when ready.
- ✓ Squash on merge if comments add noise.

MORE GITHUB

Issues. CI. Forks.

ISSUES

The team todo

Bugs, features, tasks. Templates, labels, milestones.

`"closes #42"`

in a PR auto-closes when merged.

ACTIONS

CI on push

YAML files in `.github/workflows/`. Tests, lint, build, deploy.

`"green before merge"`

is the team norm. Red = fix it.

FORKS

Open source

Fork = your own copy of someone else's repo. Work there. Send PR back.

`on a team? branch.`

contributing? fork.

SECTION 08

Things that bite.

Real-world tips that prevent disasters.

COMMON SCREWUPS

Recovery recipes.

Committed to wrong branch

```
• • •  
  
$ git log -1          # note the SHA  
$ git reset --hard HEAD~1  
$ git switch correct-branch  
$ git cherry-pick <sha>
```

Forgot to pull, push rejected

```
• • •  
  
$ git pull --rebase    # replay your commits on top  
$ git push
```

Accidentally added a huge file

```
• • •  
  
# before pushing – just amend  
$ git rm --cached huge.zip  
$ echo "huge.zip" >> .gitignore  
$ git add .gitignore  
$ git commit --amend
```

”It worked yesterday” — find when it broke

```
• • •  
  
$ git bisect start  
$ git bisect bad        # now  
$ git bisect good HEAD~20 # yesterday  
# git checks out the middle – you test, mark good/bad
```

`.GITIGNORE`

What NOT to track.

```
# .gitignore – at repo root
```

```
# dependencies
```

```
node_modules/
```

```
venv/
```

```
__pycache__/
```

```
# build outputs
```

```
dist/
```

```
build/
```

```
*.o
```

```
*.class
```

```
# environment / secrets ⚠
```

```
.env
```

```
.env.local
```

```
*.pem
```

```
# OS junk
```

```
.DS_Store
```

```
Thumbs.db
```

```
# editor
```

Set this up **day one**. Adding it later means you’ve already polluted the repo.

Pro move: use the GitHub starter for your language —

github.com/github/gitignore

already tracked it? `git rm --cached <file>`

 CRITICAL

Never commit secrets.

API keys. Passwords. Tokens. Private keys. .env files.

Once pushed, consider them **leaked**. Git history is forever — bots scrape GitHub for credentials within minutes.

leaked it? rotate the secret. NOW.

Don't try to hide it with a delete commit — the SHA still exists.

DEFENSES

- ✓ Add .env to .gitignore day one
- ✓ Commit .env.example with dummy values
- ✓ Use a secret manager (1Password, Doppler, Vault)
- ✓ Enable GitHub secret scanning
- ✓ Pre-commit hook: `git-secrets` / `trufflehog`

⚠ THE DANGEROUS ONE

Force push: handle with care.

```
• • •  
# rewrites history on the remote ✨  
$ git push --force
```

```
# better – fails if someone else pushed since you fetched  
$ git push --force-with-lease
```

```
# NEVER on shared branches:  
#   main, develop, release/*
```

```
# OK on your own branch after a rebase
```

Force push rewrites history on the server. Anyone with that branch checked out gets stranded.

Fine on your own feature branch (esp. after a rebase). **Catastrophic** on main.

always use --force-with-lease.

THE CHEATSHEET

20 commands cover 95% of daily git.

DAILY LOOP

```
git status
git diff  (--staged)
git add  (. -p)
git commit -m
git push  (-u origin)
git pull
```

BRANCHES

```
git branch
git switch -c <name>
git switch <name>
git merge <branch>
git rebase main
git branch -d <name>
```

INSPECT & UNDO

```
git log --oneline --graph
git restore <file>
git restore --staged
git revert <sha>
git stash / stash pop
git reflog ⚡
```

PRACTICE

- ✓ learngitbranching.js.org
- ✓ Build a project. Break it. Recover.
- ✓ Read your team's PRs — see what good looks like

REFERENCE

- ✓ Pro Git book — git-scm.com/book (free)
- ✓ ohshitgit.com — recovery recipes
- ✓ `git help <command>`

questions →